

PM AI Bootcamp — Project Brief Bank v2.0

Purpose: instead of assigning the same LMS case to all 300 PMs, the bootcamp can split learners across **six concrete project briefs**. Each brief is specific enough for teams to run the workbook flow end-to-end: Requirement -> WBS -> Estimation -> Risk -> Weekly report -> Delegation Map.

How to use: assign one brief per learner group and keep the same artifact pattern through the workbooks (REQ-PB01.md , WBS-PB01.md , RISK-PB01.md , and so on).

What matters most: instructors score **PM judgment**, not "how good AI sounded."

What to do after receiving a brief: follow the **Capstone Playbook** to move from brief -> clarified scope -> SW spec -> architecture -> working Customer Zero. The Playbook is the **thread that strings the 6 drills EX-01 through EX-06** into a single end-to-end flow.

Briefs are written to be easy to understand: familiar problems, compact MVP scope, mainstream technology. Each brief embeds **1-2 "clarification traps"** (intentionally vague or contradictory requirements) to train the **hard-stop** reflex in EX-01. In addition, each brief's scope is **deliberately slightly too large** for the team size and timeline — so EX-03 forces the PM to **cut scope and write a trade-off** (iteration loop).

Terminology: see docs/GLOSSARY.md — the unified glossary with a column mapping to international industry terms.

Brief index

Code	Project	Domain	Platform	Difficulty	Good fit
PB-01	Online storefront for a mid-sized shop	E-commerce	Web + backend + DB	★★☆	Web / full-stack teams
PB-02	Clinic appointment booking app	Healthcare booking	Mobile + API	★★☆	Mobile teams
PB-03	Internal warehouse management system	Internal tool	Web + DB	★☆☆	Newest groups, easiest case
PB-04	Sales reporting dashboard	Data / BI	Pipeline + dashboard	★★☆	Data teams
PB-05	Loyalty points mobile app	Customer loyalty	Mobile + backend	★★☆	Mobile / backend teams

Code	Project	Domain	Platform	Difficulty	Good fit
PB-06	AI contract summary and extraction feature	AI feature	SaaS add-on	★★★	Core track, closest to the program theme

Each brief includes:

- context,
- business goal,
- MVP scope,
- users and stakeholders,
- functional requirements,
- non-functional requirements,
- technology direction,
- team and timeline,
- constraints / sensitive data,
- clarification traps for EX-01,
- facilitator hints.

PB-01 — Online storefront for a mid-sized retail shop

Context: a fashion store currently sells through Facebook, Zalo, and manual message handling. Orders and stock are error-prone because the process is semi-manual.

Business goal: increase online orders and reduce manual order handling.

MVP scope:

- Product catalog with images, price, description, size, and color
- Search and filtering
- Cart and checkout
- Guest checkout
- Online payment plus COD
- Admin area for product and order management
- Email order confirmation

Out of scope: native mobile app, multi-language, advanced promotions, multi-branch support.

Suggested stack: Next.js + Node/NestJS or Spring Boot + PostgreSQL + local payment gateway + object storage + Docker + basic CI/CD.

Sensitive zone: payment and personal customer information.

Required clarification questions:

- What happens when online payment fails or requires refund handling?

- When exactly is stock reduced: when the order is placed or when the order is confirmed?

Facilitator notes: - WBS commonly **misses:** payment gateway integration + callback/reconciliation handling, product seed data, admin panel, sandbox payment testing. - Delegation Map: payment integration and order status changes -> **A+ (approx L4)**, human approver; product description generation / static UI -> **A (approx L3)**. - **EX-01 Stakeholder simulation:** example scope-creep request for PB-01: *"We want coupon / discount codes in MVP for the launch sale."* The correct PM response: assess timeline impact + propose "adjust prices manually for launch, coupons move to phase 2." - **EX-03 Trade-off:** PB-01 scope (online payment + admin + stock management + dashboard) will almost certainly exceed the ~130 available person-days (4 people x 8 weeks minus meetings). The PM **must cut** — grade the quality of the cut + the diff between v1 and v2. - **EX-05 Hallucination:** week-3 data will not have "revenue", "conversion rate", or "page load speed". If AI fabricates these -> PM must catch and replace with "N/A".

PB-02 — Clinic appointment booking app

Context: a small clinic receives bookings by phone. Peak periods create line congestion and overlapping doctor schedules.

Business goal: reduce manual scheduling calls and appointment conflicts.

MVP scope:

- View doctors and specialties
- View free time slots
- Book / cancel / reschedule
- Push and SMS reminders
- Reception dashboard for daily schedule
- Confirm patient arrival

Out of scope: full EMR, billing, telemedicine, insurance integration.

Suggested stack: Flutter or React Native + backend API + PostgreSQL + FCM + SMS/OTP provider + web dashboard for reception.

Sensitive zone: patient data, OTP, schedule correctness.

Required clarification questions:

- Is booking instant, or does reception manually approve each booking?
- How many reminder messages are allowed, and what is the SMS budget rule?

Facilitator notes: - WBS commonly misses: booking slot concurrency handling, SMS/OTP integration and cost, testing on real devices. - Delegation Map: login/OTP, sending SMS to patients -> **A+ (approx L4)**; generating static screens, notification text -> **A (approx L3)**. - **EX-01 Stakeholder simulation:** example scope-creep request: *"The doctor wants to see the patient's medical history right in the app — it would be convenient during appointments."* PM should respond: medical records = out-of-scope for MVP + sensitive health data -> phase 2,

requires compliance assessment first. - **EX-03 Trade-off:** mobile app + backend + SMS/OTP + reception web dashboard -> 5 people x 8 weeks is tight. PM should cut or simplify (e.g. drop SMS reminders, use push notifications only as a cheaper alternative). - **EX-05 Hallucination:** watch for AI fabricating "no-show rate" or "booking volume" when no operational data exists yet.

PB-03 — Internal warehouse management system

Context: inventory is currently controlled by Excel. Real stock is hard to trust and manual corrections are frequent.

Business goal: maintain accurate stock levels and audit-friendly movement history.

MVP scope:

- Product catalog
- Goods receipt
- Goods issue
- Current stock view
- Movement history
- Stock report by period
- Two roles: warehouse clerk and warehouse manager

Out of scope: multi-warehouse, barcode devices, accounting integration, demand forecast.

Suggested stack: React or Vue + Spring Boot or .NET + MySQL or PostgreSQL + Excel export.

Sensitive zone: permissions and manual stock correction.

Required clarification questions:

- How are inventory adjustments handled when physical stock and recorded stock differ?
- Who is allowed to edit master product data?

Facilitator notes: - Easiest brief -> good for teaching clean WBS and estimation. Common misses: permission screens, audit logs, stock adjustment workflow. - Delegation Map: permission changes and manual stock adjustments -> **A+ (approx L4)**, human approver; CRUD catalog, read-only reports -> **A (approx L3)** or lower. - **EX-01 Stakeholder simulation:** example scope-creep request: *"The warehouse wants barcode scanning from a phone."* PM should respond: barcode scanning = hardware scope + mobile app -> out of scope for a 6-week MVP, log for phase 2. - **EX-03 Trade-off:** 3 people x 6 weeks — scope is compact but PMs often forget audit logs + permissions; if estimates exceed capacity -> cut Excel export (use CSV instead). - **EX-05 Hallucination:** AI tends to fabricate "permission error rate" or "number of stock adjustments" when no data exists -> PM must catch and mark N/A.

PB-04 — Sales reporting dashboard

Context: leadership currently receives separate Excel reports from branches and cannot trust that the numbers reconcile.

Business goal: provide one daily decision dashboard with a single version of truth.

MVP scope:

- Pull sales data from one API-based system and two Excel-based branches
- Clean and normalize the data
- Centralized storage
- Dashboard with time, branch, product-group, and comparison views
- Nightly refresh
- Error notification when a source fails

Out of scope: ML forecasting, real-time minute-level analytics, individual employee performance analytics.

Suggested stack: Python ETL or equivalent + warehouse / PostgreSQL + BI tool or dashboard web app + scheduler + run logs.

Sensitive zone: revenue data and source trustworthiness.

Required clarification questions:

- The brief says sources are "already standardized", but the branch Excel templates are actually inconsistent. What is the true cleaning scope?
- If one branch is late, what should the dashboard show?

Facilitator notes: - This is the best brief for teaching **EX-05 (anti-rubber-stamp and AI data fabrication)**: require PM to attach a source to every KPI and flag "data missing". - WBS commonly misses: data cleaning / product code reconciliation across sources, error-source handling, total reconciliation step. - Delegation Map: pipeline writing to the executive data warehouse -> **A+ (approx L4)** with a reconciliation gate; generating draft queries / charts -> **A (approx L3)**. - **EX-01 Stakeholder simulation**: example scope-creep request: *"The boss wants a real-time dashboard refreshing every minute."* PM should respond: real-time requires streaming infrastructure, out of scope for an 8-week MVP — nightly batch before 7 AM already meets leadership decision needs. - **EX-03 Trade-off**: ETL + cleaning 3 sources + dashboard + reconciliation -> 4 people x 8 weeks is tight. PM should cut: drop custom-built dashboard, use an off-the-shelf tool (Metabase / Superset) instead. - **EX-05 Hallucination**: the "anchor" brief for EX-05 — AI tends to fabricate "projected revenue" or interpolate numbers when source data is missing. PM must force the dashboard to show "N/A — source not submitted" instead of letting AI fill in.

PB-05 — Loyalty points mobile app

Context: a retail chain wants a lightweight customer loyalty app so returning customers can collect and redeem points instead of using paper cards.

Business goal: improve repeat purchases and capture basic customer behavior.

MVP scope:

- Registration / login
- View points balance
- Earn points from purchases
- Redeem points for simple rewards
- Notifications for point events
- Basic admin view for reward rules and manual adjustment

Out of scope: full CRM suite, deep segmentation, omnichannel analytics, complex coupon engines.

Suggested stack: Flutter or React Native + backend API + PostgreSQL + push notifications.

Sensitive zone: customer identity, reward abuse, manual point adjustment.

Required clarification questions:

- How is purchase proof captured for adding points?
- Who is allowed to manually adjust customer balances?

Facilitator notes: - WBS commonly misses: duplicate scan prevention, cashier app/screen, point rule configuration, real-device QR testing. - Delegation Map: mass push notifications, point redemption (balance deduction), rule configuration -> **A+ (approx L4)**, human approver; UI generation / draft notification content -> **A (approx L3)**. - **EX-01 Stakeholder simulation:** example scope-creep request: *"Marketing wants combo-based point earning (buy 2 get 1 -> bonus points)."* PM should respond: combo logic is complex, out of scope for MVP — phase 1 needs only a single rule (X points per Y currency spent). - **EX-03 Trade-off:** 5 people x 8 weeks but 2 apps (customer + cashier) + backend + push + OTP -> tight. PM should cut: drop customer segmentation for push (send to all or manual select), or merge cashier screen into web instead of a separate app. - **EX-05 Hallucination:** AI tends to fabricate "redemption rate" or "customer return rate" when no operational data exists yet — PM must catch and mark N/A.

PB-06 — AI contract summary and extraction feature

Context: account and legal teams spend too much time reading customer contracts and extracting key obligations by hand.

Business goal: shorten contract review time while keeping legal review quality intact.

MVP scope:

- Upload a contract document
- Extract predefined fields (customer name, dates, commercial terms, obligations, exceptions)
- Generate a summary
- Highlight uncertain fields
- Keep a review screen for human correction

- Save reviewed output with audit trail

Out of scope: full autonomous legal approval, multilingual legal reasoning, broad enterprise repository integration.

Suggested stack: SaaS app extension or lightweight web app + OCR if needed + LLM pipeline + backend API + structured storage + audit log.

Sensitive zone: legal data, privacy, hallucination, AI overconfidence.

Required clarification questions:

- May the contract be sent to an external model API, or must it stay inside a controlled environment?
- Which fields are mandatory, and how is confidence shown to the reviewer?

Facilitator notes (anchor brief for the program): - This brief forces the PM to apply **the full CASAN/Leash philosophy**: humans hold the decision, AI is the execution force; AI output passes through a **fail-closed gate** (traceable to source before use); watch for **rubber-stamping** (blindly approving AI results). - WBS commonly misses: extraction quality evaluation step, PDF scan/OCR handling, source citation display mechanism, data security testing. - Delegation Map (most critical for this brief): - Sending contracts to an external model, saving results to the official record -> **A+ (approx L4)** + security gate + human approver. - AI generating a draft summary for human editing -> **A (approx L3)**. - **Never** let AI autonomously "confirm" a legal field without human review (this illustrates why **L5 is never granted**). - **EX-01 Stakeholder simulation**: example scope-creep request: *"The client wants AI to assess legal risk — 'is this contract safe?'"*. PM should respond: legal advice = out of scope + serious liability risk -> extraction only, **no judgment**. - **EX-03 Trade-off**: AI engineer + backend + frontend + OCR + RAG -> 4 people x 8 weeks is tight. PM should cut: drop OCR support (only support text-based PDFs, not scanned documents) — saves approximately 2 weeks of effort. - **EX-05 Hallucination**: the "anchor" brief for hallucination detection — AI will fabricate penalty clauses or infer legal meaning from ambiguous passages. PM must enforce: field not found -> "Not found — manual review required", **never let AI fill in with guesses**.

Rollout guidance for the 300 PM program

- Use **PB-03** for new groups.
- Use **PB-06** for the strongest core groups.
- Mix briefs by team profile so different roles see different risk types.
- Keep artifact naming consistent by brief code.
- The same six briefs can be reused across PM / BA / SA / Dev tracks by changing the expected role-specific artifact.

The brief bank exists to make the program concrete, comparable, and scalable without forcing all learners into a single domain scenario.

Terminology: for full definitions and international-standard mappings, see [docs/GLOSSARY.md](#).

Appendix — Changelog

Version	Date	Changes
v1.0	2026-06-30	Initial set of 6 project briefs (PB-01 through PB-06) — detailed requirements + clear technology direction, mapped to EX-01 through EX-06.
v2.0	2026-08-24	Added instructor notes for EX-01 stakeholder simulation, EX-03 trade-off, and EX-05 hallucination detection for every brief (PB-01 through PB-06). Added glossary pointer. Changelog moved to end.

PM AI Bootcamp v2.0 — Project Brief Bank -- 2026-08-24 -- Use alongside Workbook Common v2.0.